
Department of Computer Science & Engineering Education



Information Retrieval Techniques

NAME - RAJ

ENROLLMENT NO. - NITRB-25-BD-PG-010

COURSE - M.TECH CSE(BIG DATA ANALYTICS)

GithubLink

<https://github.com/rajsanodiya122/Information Retrieval Techniques>

Unit-1&2

Activity 4 : Smart Job Search System using Text Processing and Query Expansion

Problem Statement

Modern job portals such as LinkedIn, Naukri, and Indeed host thousands of job postings with detailed information including skills, responsibilities, qualifications, and company details. However, users often search using simple queries such as:

- “AI job”
- “data analyst”
- “software role in cloud”

Due to the mismatch between user queries and structured job data fields (like Skills, Role, Job Description), traditional keywordbased systems fail to retrieve relevant job postings. To solve this problem, an intelligent retrieval system must:

- Process multiple textual fields
- Represent documents effectively
- Rank results based on relevance
- Improve search using semantic query expansion

Objective

Develop a job retrieval system that:

- Uses multiple dataset fields for indexing
- Implements TF-IDF for document representation
- Retrieves and ranks jobs using cosine similarity
- Enhances search results using Query Expansion (WordNet)
- Evaluates improvement in retrieval performance

Dataset

Students must use the provided Kaggle Job Dataset with fields such as:

- Job Title
- Role
- Job Description
- Skills
- Responsibilities

- Company Name
- Location

<https://www.kaggle.com/datasets/ravindrasinghrana/job-description-dataset>

Method / Approach Used

The system was implemented using a Kaggle Job Description dataset. Important fields such as Job Title, Role, Skills, Job Description, and Responsibilities were combined into a single column called **content** to improve search coverage.

Text preprocessing was applied, including:

- Lowercasing
- Tokenization
- Stopword removal using NLTK
- Removal of special characters

For document representation, the TF-IDF technique was used to convert textual data into numerical vectors. When a user enters a query, it is transformed into a TF-IDF vector, and similarity is computed using Cosine Similarity. Jobs are then ranked based on similarity scores, and the top 5 results are displayed with job title, company name, and relevance score.

To further enhance retrieval, query expansion was implemented using WordNet. Synonyms of query terms were extracted and appended to the original query, enabling the system to capture semantic meaning rather than relying only on exact keywords.

Screenshots

Dataset Loading

`df.head()` and `df.columns`

```
▶ import pandas as pd
import numpy as np
import nltk
import re

from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
[4] ✓ 1s df = pd.read_csv("/content/job_descriptions.csv", engine='python', on_bad_lines='warn')

print(df.head())
print(df.columns)
```

	343.975.4702x9340	Event Manager
0	Social Media Manager	Snagajob
1	Frontend Web Developer	Idealist
2	Quality Control Manager	Jobs2Careers
3	Wireless Network Engineer	FlexJobs
4	Conference Manager	Jobs2Careers

	Job Description
0	Social Media Managers oversee an organizations...
1	Frontend Web Developers design and implement u...
2	Quality Control Managers establish and enforce...
3	Wireless Network Engineers design, implement, ...
4	A Conference Manager coordinates and manages c...

	Benefits
0	{'Flexible Spending Accounts (FSAs), Relocatio...
1	{'Health Insurance, Retirement Plans, Paid Tim...
2	{'Legal Assistance, Bonuses and Incentive Prog...
3	{'Transportation Benefits, Professional Develo...
4	{'Flexible Spending Accounts (FSAs), Relocatio...

	skills
0	Social media platforms (e.g., Facebook, Twitte...
1	HTML, CSS, JavaScript Frontend frameworks (e.g...
2	Quality control processes and methodologies St...
3	Wireless network design and architecture Wi-Fi...
4	Event planning Conference logistics Budget man...

	Responsibilities
0	Manage and grow social media accounts, create ...
1	Design and code user interfaces for websites, ...
2	Establish and enforce quality control standard...
3	Design, configure, and optimize wireless netwo...
4	Specialize in conference and convention planni...

	Company
0	Icahn Enterprises
1	PNC Financial Services Group

Text preprocessing techniques were applied to clean the dataset. These included:

- * Converting text into lowercase
- * Removing special characters
- * Tokenization
- * Stopword removal using NLTK

Content Creation

```
print(df[['Job Title','content']].head())
```

```

0      Company \
1      Icahn Enterprises
2      PNC Financial Services Group
3      United Services Automobile Assn.
4      Hess
5      Cairn Energy

Company Profile
0 {"Sector":"Diversified","Industry":"Diversifie...
1 {"Sector":"Financial Services","Industry":"Com...
2 {"Sector":"Insurance","Industry":"Insurance: P...
3 {"Sector":"Energy","Industry":"Mining, Crude-O...
4 {"Sector":"Energy","Industry":"Energy - Oil & ...

[5 rows x 23 columns]
Index(['Job Id', 'Experience', 'Qualifications', 'Salary Range', 'location',
      'Country', 'latitude', 'longitude', 'Work Type', 'Company Size',
      'Job Posting Date', 'Preference', 'Contact Person', 'Contact',
      'Job Title', 'Role', 'Job Portal', 'Job Description', 'Benefits',
      'skills', 'Responsibilities', 'Company', 'Company Profile'],
      dtype='object')

[7]
✓ 0s df['content'] = (
    df['Job Title'].fillna('') + " " +
    df['Role'].fillna('') + " " +
    df['Job Description'].fillna('') + " " +
    df['skills'].fillna('') + " " +
    df['Responsibilities'].fillna('')
)
print(df[['Job Title', 'content']].head())

Job Title \
0 Digital Marketing Specialist
1 Web Developer
2 Operations Manager
3 Network Engineer
4 Event Manager

content
0 Digital Marketing Specialist Social Media Mana...
1 Web Developer Frontend Web Developer Frontend ...
2 Operations Manager Quality Control Manager Qua...
3 Network Engineer Wireless Network Engineer Wir...
4 Event Manager Conference Manager A Conference ...

```

Preprocessing Output

```
print(df[['content','cleaned_content']].head())
```

[26]
✓ 15s

```
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()

    # remove special characters
    text = re.sub(r'^a-zA-Z\s', '', text)

    # tokenize
    tokens = word_tokenize(text)

    # remove stopwords
    tokens = [word for word in tokens if word not in stop_words]

    return " ".join(tokens)

df['cleaned_content'] = df['content'].apply(preprocess)

print(df[['content', 'cleaned_content']].head())
```

```
content \
0 Digital Marketing Specialist Social Media Mana...
1 Web Developer Frontend Web Developer Frontend ...
2 Operations Manager Quality Control Manager Qua...
3 Network Engineer Wireless Network Engineer Wir...
4 Event Manager Conference Manager A Conference ...

cleaned_content
0 digital marketing specialist social media mana...
1 web developer frontend web developer frontend ...
2 operations manager quality control manager qua...
3 network engineer wireless network engineer wir...
4 event manager conference manager conference ma...
```

After preprocessing, TF-IDF (Term Frequency-Inverse Document Frequency) was used to convert textual job descriptions into numerical vectors.

When a user entered a query, the query was also converted into a TF-IDF vector. Cosine similarity was calculated between the query vector and all job vectors to measure relevance.

TF-IDF Search Output

Query results showing top 5 jobs.

event manager conference manager conference ma...

[12]
✓ 3s

#Task 2: TF-IDF

```
vectorizer = TfidfVectorizer()  
tfidf_matrix = vectorizer.fit_transform(df['cleaned_content'])
```

[10]
✓ 14s

```
query = input("Enter job search query: ")
```

Enter job search query: Ai job

[13]
✓ 0s

```
query_clean = preprocess(query)  
query_vector = vectorizer.transform([query_clean])
```

[14]
✓ 0s

```
similarity_scores = cosine_similarity(query_vector, tfidf_matrix)
```

[15]
✓ 0s

```
scores = similarity_scores.flatten()  
top_indices = scores.argsort()[-5:][::-1]
```

[17]
✓ 0s

```
print("Top 5 Matching Jobs:\n")  
  
for i in top_indices:  
    print("Job Title:", df.iloc[i]['Job Title'])  
    print("Company Name:", df.iloc[i]['Company'])  
    print("Score:", scores[i])  
    print("-"*50)
```

Top 5 Matching Jobs:

```
Job Title: HR Generalist  
Company Name: Meta Platforms  
Score: 0.121805145190331  
-----  
Job Title: HR Generalist  
Company Name: Fidelity National Information Services  
Score: 0.121805145190331
```



```
[17]
✓ Os
print("Top 5 Matching Jobs:\n")

for i in top_indices:
    print("Job Title:", df.iloc[i]['Job Title'])
    print("Company Name:", df.iloc[i]['Company'])
    print("Score:", scores[i])
    print("-"*50)

Top 5 Matching Jobs:

Job Title: HR Generalist
Company Name: Meta Platforms
Score: 0.121805145190331
-----
Job Title: HR Generalist
Company Name: Fidelity National Information Services
Score: 0.121805145190331
-----
Job Title: HR Generalist
Company Name: Sprint Corporation
Score: 0.121805145190331
-----
Job Title: HR Generalist
Company Name: Wesfarmers Limited
Score: 0.121805145190331
-----
Job Title: HR Generalist
Company Name: Petronet LNG
Score: 0.121805145190331
-----
```

A basic job search system was developed to rank job postings and display the top five most relevant jobs.

Basic Search

```
[ ]
#Task 3: Build Basic Search System

[20]
✓ Os
def basic_search(query):
    query_clean = preprocess(query)

    query_vector = vectorizer.transform([query_clean])

    similarity = cosine_similarity(query_vector, tfidf_matrix)

    scores = similarity.flatten()

    top_indices = scores.argsort()[-5:][::-1]

    print("\nTop 5 Search Results:\n")

    for i in top_indices:
        print("Job Title:", df.iloc[i]['Job Title'])
        print("Company Name:", df.iloc[i]['Company'])
        print("Location:", df.iloc[i]['location'])
        print("Score:", round(scores[i],4))
        print("-"*50)
```

To improve search performance, query expansion was implemented using WordNet. Synonyms of query terms were extracted and added to the original query to retrieve semantically related job postings.

Query Expansion Output

```
[1] #Task 4: Query Expansion using WordNet

[24] ✓ 0s def expand_query(query):
    expanded_terms = []

    for word in query.split():
        expanded_terms.append(word)

        synsets = wordnet.synsets(word)

        for syn in synsets[:2]:
            for lemma in syn.lemmas()[:2]:
                synonym = lemma.name().replace("_", " ")
                expanded_terms.append(synonym)

    expanded_query = " ".join(set(expanded_terms))

    return expanded_query
def search_with_expansion(query):
    expanded_query = expand_query(query)

    print("Original Query:", query)
    print("Expanded Query:", expanded_query)

    query_clean = preprocess(expanded_query)

    query_vector = vectorizer.transform([query_clean])

    similarity = cosine_similarity(query_vector, tfidf_matrix)

    scores = similarity.flatten()

    top_indices = scores.argsort()[-5:][::-1]

    print("\nTop 5 Results After Query Expansion:\n")

    for i in top_indices:

        for i in top_indices:
            print("Job Title:", df.iloc[i]['Job Title'])
            print("Company Name:", df.iloc[i]['Company'])
            print("Location:", df.iloc[i]['location'])
            print("Score:", round(scores[i],4))
            print("-"*50)
```

Result Comparison

* Without Query Expansion

* With Query Expansion

22]
r 0s

#Task 5 Result

25]
r 0s

```
print("WITHOUT QUERY EXPANSION")
basic_search("AI job")
```

```
print("\nWITH QUERY EXPANSION")
search_with_expansion("AI job")
```

✓

```
Job Title: HR Generalist
Company Name: Meta Platforms
Location: Caracas
Score: 0.1218
-----
```

```
Job Title: HR Generalist
Company Name: Fidelity National Information Services
Location: Ljubljana
Score: 0.1218
-----
```

```
Job Title: HR Generalist
Company Name: Sprint Corporation
Location: Dili
Score: 0.1218
-----
```

```
Job Title: HR Generalist
Company Name: Wesfarmers Limited
Location: Mbabane
Score: 0.1218
-----
```

```
Job Title: HR Generalist
Company Name: Petronet LNG
Location: Gaza
Score: 0.1218
-----
```

WITH QUERY EXPANSION

Original Query: AI job

Expanded Query: occupation business artificial intelligence Army Intelligence task job AI

Top 5 Results After Query Expansion:



WITH QUERY EXPANSION

Original Query: AI job

Expanded Query: occupation business artificial intelligence Army Intelligence task job AI

Top 5 Results After Query Expansion:

Job Title: Data Analyst

Company Name: Comcast

Location: Dushanbe

Score: 0.357

Job Title: Data Analyst

Company Name: Massachusetts Mutual Life Insurance

Location: Brasilia

Score: 0.357

Job Title: Data Analyst

Company Name: Aurobindo Pharma

Location: New Delhi

Score: 0.357

Job Title: Data Analyst

Company Name: ServiceNow

Location: Guatemala City

Score: 0.357

Job Title: Data Analyst

Company Name: Freeport-McMoRan

Location: Warsaw

Score: 0.357

Observations

Without query expansion, the system retrieved only jobs containing exact terms like “AI.” This limited the results significantly. After applying query expansion, the system was able to retrieve related roles such as Machine Learning Engineer, Data Scientist, and NLP Engineer.

This resulted in:

- Increased number of relevant results
- Better ranking quality
- Discovery of hidden job opportunities

Thus, query expansion significantly improved retrieval effectiveness.

Challenges

Key challenges faced during implementation included:

- Handling missing and inconsistent data
- Column mismatches (e.g., Skills vs skills)
- Noisy and unstructured text
- Irrelevant synonyms generated by WordNet

These were resolved through preprocessing, data cleaning, and filtering techniques.

Conclusion

The project demonstrates how Natural Language Processing techniques can enhance job search systems. TF-IDF and cosine similarity provided an effective baseline for ranking, while query expansion improved semantic understanding and retrieval performance. Compared to traditional keyword-based systems, this approach delivers more accurate and meaningful job recommendations, making it highly useful for modern recruitment platforms.